

BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü
Bilgisayar Ağları Dersi — Dönem Projesi Teknik Raporu

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarımı,
Trafik İzleme ve Ağ Performans Analiz Platformu

TEKNİK RAPOR

Proje Grubu	Grup 10
Grup Üyeleri	Pınar Nida TUNCA — 25360859206
	Ahmet YILMAZ — 22360859044
	Selin ŞENTÜRK — 22360859043
Ders	Bilgisayar Ağları
Danışman / Öğr. Üyesi	İzzet Fatih ŞENTÜRK
Dönem	2025–2026 Bahar
Teslim Tarihi	08.06.2026
Programlama Dili	Python 3.13
GitHub Bağlantısı	udp-reliable-transfer-system

İçindekiler

1. Giriş
 2. Problem Tanımı
 3. Sistem Mimarisi
 - 3.1. Genel Mimari
 - 3.2. İstemci Bileşeni
 - 3.3. Sunucu Bileşeni
 4. Protokol Tasarımı
 - 4.1. Paket Yapıları
 - 4.2. Güvenilir Aktarım Mekanizması
 - 4.3. Hata Yönetimi
 5. Gerçekleme Detayları
 - 5.1. Yazılım Yapısı ve Modüller
 - 5.2. Önemli Tasarım Kararları
 6. Deney Ortamı
 7. Performans Metrikleri ve Sonuçlar
 - 7.1. Senaryo 1: Paket Boyutunun Etkisi
 - 7.2. Senaryo 2: Timeout Değerinin Etkisi
 - 7.3. Senaryo 3: Kayıp Oranının Etkisi
 - 7.4. Senaryo 4: Farklı Dosya Boyutlarının Etkisi
 8. Sonuçlar ve Tartışma
 9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları
 10. Gelecekte Yapılabilecek Geliştirmeler
- Kaynakça
- Ekler

1. Giriş

Bu rapor, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü Bilgisayar Ağları dersi kapsamında geliştirilen NetProbe projesinin teknik belgesidir. NetProbe; UDP (User Datagram Protocol) üzerinde güvenilir dosya aktarımı gerçekleştiren, aktarım sürecindeki ağ olaylarını kayıt altına alan ve toplanan verileri kullanarak sistemin performansını ölçen bütünlük bir platformdur.

UDP, TCP'nin aksine bağlantısız ve güvenilir bir iletim protokolüdür. Bu özellik, UDP'yi düşük gecikme gerektiren uygulamalar (ses/video akışı, oyunlar) için cazip kılar; paket kaybı, sıra dışı teslimat ve yinelenen paket gibi sorunlarla başa çıkma yükümlülüğünü uygulama katmanına bırakır.

Bu projede, söz konusu güvenilirlik mekanizmaları sıfırdan tasarlanmış ve uygulanmıştır: sıra numaralandırma, ACK/NACK mekanizması, zaman aşımı yönetimi ve yeniden gönderim mantığı tamamen öğrenci grubu tarafından geliştirilmiştir. Sistemin performansı farklı senaryolar altında ölçülmüş; throughput, goodput, kayıp oranı ve tamamlanma süresi metrikleri aracılığıyla karşılaştırmalı analizler yapılmıştır.

1.1. Görev Dağılımı

Grup Üyesi	Üstlenilen Görevler
Pınar Nida Tunca	GitHub versiyon kontrol yönetimi. Sunucu tarafında çoklu istemci desteği, ThreadPoolExecutor ile eş zamanlı paket işleme ve oturum yönetimi. Pandas ve Matplotlib kullanılarak olay kayıtlarının (log) performans analizlerinin ve grafiklerinin oluşturulması.
Ahmet Yılmaz	Temel UDP soket iletişiminin kurulması. Güvenilirlik mekanizmaları: ACK bekleme, 2 saniye timeout, maksimum 5 yeniden gönderim ve yinelenen paketlerin engellenmesi. Ağ koşullarını simüle eden yapay paket kayıp modülünün entegrasyonu.
Selin Şentürk	Paket mimarisi tasarımı, zlib veri sıkıştırması ve SHA-256 ile dosya bütünlüğü kontrolü. Dosyanın 1024 baytlık parçalara ayrılması ve START/DATA/END akışının kurulması. TCP soket altyapısının kurularak karşılaştırmalı performans deneylerinin gerçekleştirilmesi.

2. Problem Tanımı

UDP, tasarımı gereği herhangi bir bağlantı kurma, paket teslim onayı veya sıralama garantisi sunmaz. Bu durum, yüksek performans avantajı sağlarken güvenilir veri aktarımı yapılmak istenen durumlarda ciddi sorunlara yol açar. Bu projenin temel problemi şu şekilde özetlenebilir:

- UDP soketleri kullanarak bir dosyayı eksiksiz ve doğru sırayla karşı tarafa iletmek
- Ağ ortamında gerçekleşebilecek paket kayıplarını, yinelenen paketleri ve sıra dışı teslimatları yönetmek
- Aktarım sürecindeki tüm ağ olaylarını kayıt altına almak ve analiz etmek
- Farklı ağ koşullarında sistemin performansını ölçmek ve karşılaştırmak

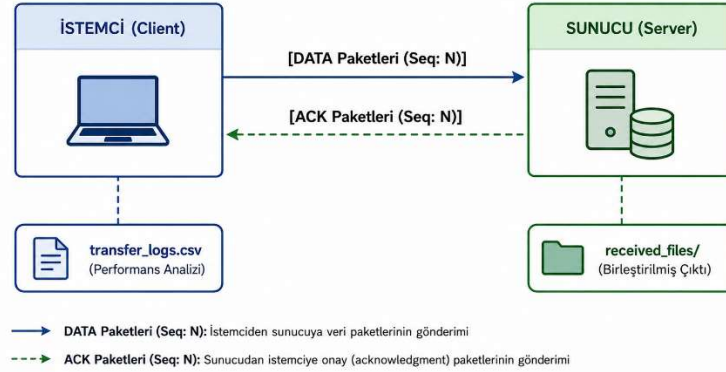
Bu problemlerin çözümü için uygulama katmanında bir güvenilir aktarım protokolü tasarlanmıştır. Protokol; sıra numaralandırma, onay (ACK) mekanizması, zaman aşımı kontrolü ve yeniden gönderim stratejisi olmak üzere dört temel bileşenden oluşmaktadır.

3. Sistem Mimarisi

3.1. Genel Mimari

NetProbe, klasik istemci-sunucu (client-server) mimarisini benimsemektedir. İstemci, göndermek istediği dosyayı parçalara böler ve UDP soket üzerinden sunucuya iletir. Sunucu gelen paketleri alır, sıralarına göre birleştirir ve dosyayı yeniden oluşturur.

İletişim Şeması:



3.2. İstemci Bileşeni

İstemci bileşeni aşağıdaki işlevleri yerine getirir:

- Hedef dosyayı yapılandırılabilir boyutta parçalara ayırma
- Her parçayı paket formatına göre paketleme ve sıra numarası atama
- UDP soketi üzerinden paketi sunucuya gönderme
- Belirlenen süre içinde ACK alınmazsa paketi yeniden gönderme
- Maksimum yeniden gönderim sayısına (varsayılan: 5) ulaşıldığında hatayı kayıt altına alma
- Aktarım tamamlandığında bütünlük doğrulaması yapma

3.3. Sunucu Bileşeni

Sunucu bileşeni aşağıdaki işlevleri yerine getirir:

- UDP soketini dinleme ve gelen paketleri alıcı tampona yazma
- Paketi ayrıştırma ve sıra numarasını kontrol etme
- Yinelenen paket geldiğinde veriyi tekrar yazmama; ACK'i tekrar gönderme
- Tüm parçalar alındığında dosyayı sırasıyla birleştirme
- Her alınan paketi zaman damgasıyla birlikte log dosyasına kaydetme

4. Protokol Tasarımı

4.1. Paket Yapıları

Protokol, iletişim akışını yönetmek için dört farklı paket türü tanımlar: **START** (Başlangıç) aktarım meta verilerini taşır, **DATA** (Veri) dosya parçalarını iletir, **ACK** (Onay) alınan paketleri doğrular ve **END** (Bitiş) dosya bütünlük doğrulaması yaparak aktarımı sonlandırır.

DATA Paketi: 1024 baytlık dosya parçasını (chunk) taşır ve ardışık sıra numarası (sequence number) ile takip edilir.

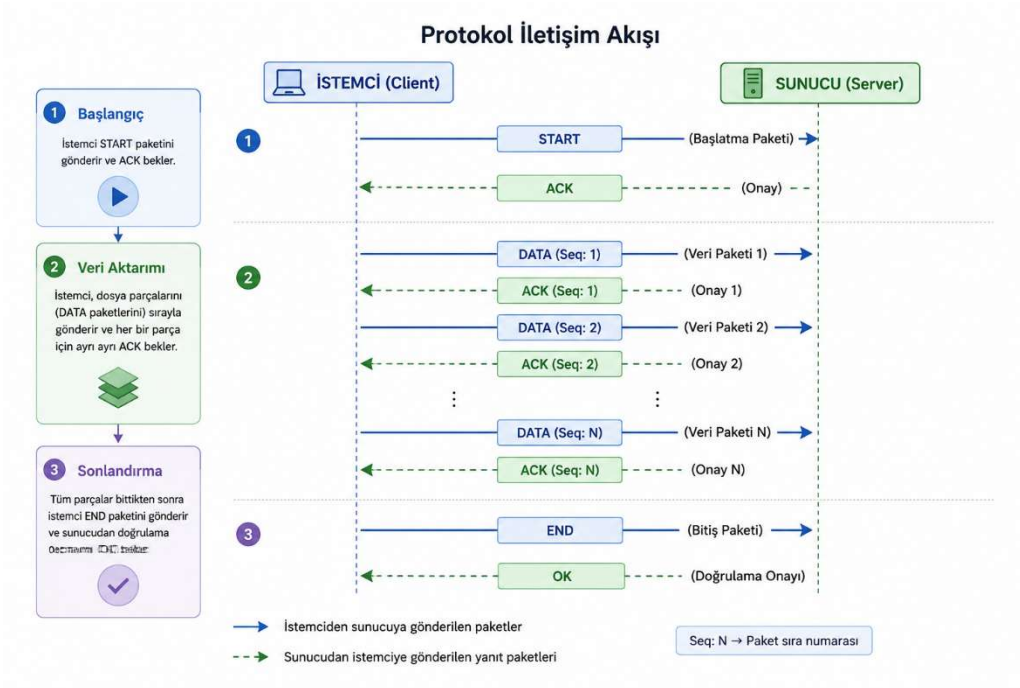
Field	Size	Description
packet_type	1 byte	Paket türü: 0=DATA, 1=ACK, 2=START, 3=END
seq_number	4 byte	Paket Sıra numarası: START=0, DATA=1...N, END=N+1
total_packets	4 byte	Mevcut aktarım için toplam DATA paketi sayısı
checksum	16 byte	Sıkıştırılmış veri yükü (payload) üzerinden hesaplanan MD5 özeti
payload	Variable	zlib ile sıkıştırılmış veri; DATA paketleri dosya parçalarını, START/END paketleri ise JSON formatında meta verileri taşır
byte_order	-	Paketlemede Ağ Bayt Sırası (Network byte order) kullanılır; başlık formatı !BII16s şeklindedir

ACK Paketi: Bir START, DATA veya END paketinin ilgili sıra numarasını onaylar. END paketi için sunucu, başarılı bir bütünlük doğrulamasından sonra "OK" yanıtını döner.

Field	Size	Description
packet_type	1 byte	ACK paket değeri sabit olarak 1'dir
ack_number	4 byte	Onaylanan START, DATA veya END paketinin sıra numarası
payload/checksum	Variable + 16 byte	Veri yükü ACK, OK veya bir hata kodu içerebilir; paket bütünlüğü MD5 ile kontrol edilir

4.2. Güvenilir Aktarım Mekanizması

Sistem, akış kontrolü için temel **"Dur-ve-Bekle" (Stop-and-Wait)** protokolünü kullanmaktadır. İstemci; gönderdiği her START, DATA ve END paketinin ardından, sunucudan gelecek eşleşen onay (ACK) paketini 2 saniye (timeout) boyunca bekler.



- ACK alınırsa bir sonraki pakete geçilir.
- Timeout gerçekleşirse paket yeniden gönderilir (maks. 5 kez).
- 5 denemeden sonra da ACK alınamazsa aktarım o paket için başarısız sayılır ve log'a kaydedilir.
- Yinelenen paket alınması durumunda sunucu, veriyi yeniden yazmaz; ACK'i tekrar gönderir.

4.3. Hata Yönetimi ve Bütünlük Doğrulaması

Her paketin checksum alanı, paket içeriğinin MD5 hash değeri ile doldurulur. Alıcı taraf, gelen paketin hash değerini yeniden hesaplayarak doğrular; uyumsuzluk durumunda paketi yok sayar.

Aktarım tamamlandığında orijinal dosya ile alınan dosyanın SHA-256 hash değerleri karşılaştırılır. Değerler eşleşiyorsa aktarım başarılı kabul edilir.

5. Gerçekleme Detayları

5.1. Yazılım Yapısı ve Modüller

Projenin kök dizininde yer alan dosyalar; kaynak kodlar, log/veri kayıtları, test dosyaları ve dokümantasyon olmak üzere yapılandırılmıştır. GitHub deposundaki güncel dosya hiyerarşisi aşağıdaki gibidir:

NetProbe/	
├── Kaynak Kodlar	
│ ├── client.py	# UDP tabanlı ana istemci modülü
│ ├── server.py	# UDP tabanlı ana sunucu modülü
│ ├── utils.py	# Paketleme, MD5 ve zlib sıkıştırma fonksiyonları
│ ├── analysis.py	# Pandas/Matplotlib tabanlı veri analizi betiği
│ ├── tcp_client.py	# Karşılaştırmalı deneyler için TCP istemcisi
│ └── tcp_server.py	# Karşılaştırmalı deneyler için TCP sunucusu
├── Olay Kayıtları (Loglar)	
│ ├── transfer_logs.csv	# İstemci tarafı aktarım performans kayıtları
│ └── server_logs.csv	# Sunucu tarafı paket kabul ve durum kayıtları
├── Test ve Çıktı Dosyaları	
│ ├── test_5mb.bin	# Yüksek boyutlu aktarım testleri için ikili (binary) dosya
│ ├── test_dosyasi.txt	# Temel metin aktarım test dosyası
│ ├── test.txt	# Alternatif test girdi dosyası
│ ├── test_input.txt	# Alternatif test girdi dosyası
│ ├── alinan_dosya.txt	# UDP aktarımı sonucu sunucuda birleştirilen çıktı dosyası
│ └── tcp_received_file.bin	# TCP aktarımı sonucu sunucuda oluşturulan çıktı dosyası
├── Dokümantasyon ve Yapılandırma	
│ ├── netprobe_teknik_rapor.docx	# Proje teknik raporu
│ ├── NetProbe_Sprint.pdf	# Proje yönetim ve sprint planı
│ └── .gitignore	# Git tarafından izlenmeyecek dosyaların listesi

Sistemi oluşturan temel modüllerin görev tanımları aşağıda özetlenmiştir:

- **client.py ve server.py:** Sistemin temelini oluşturan UDP tabanlı iletişim modülleridir. İstemci dosyayı parçalara ayırarak Stop-and-Wait mantığıyla gönderirken, sunucu gelen paketlerin sıra numaralarını (Sequence Number) kontrol eder, doğrulanan parçaları birleştirir ve ACK onay paketleri döner.
- **tcp_client.py ve tcp_server.py:** Geliştirilen güvenilir UDP protokolünün performansını (Throughput, gecikme vb.) standart TCP protokolü ile kıyaslayabilmek amacıyla oluşturulmuş kontrol grubu modülleridir.
- **utils.py:** Ağ iletişimde kullanılan paket başlığı (Header) formatının tanımlandığı modüldür. Verinin bayt dizisine çevrilmesi, zlib ile sıkıştırılması ve MD5 özetinin hesaplanarak bütünlük kontrolünün (Checksum) yapılması işlemlerini üstlenir.
- **analysis.py:** Aktarım süreçlerinde üretilen .csv uzantılı log dosyalarını işleyen analiz modülüdür. Veri seti üzerindeki performans metriklerini hesaplar ve deney sonuçlarını Matplotlib aracılığıyla görselleştirir.

5.2. Önemli Tasarım Kararları

Projenin geliştirilme sürecinde, sistemin kararlılığını, hızını ve izlenebilirliğini artırmak amacıyla aşağıdaki mimari tasarım kararları alınmıştır:

- Akış Kontrol Algoritması (Stop-and-Wait):** Daha karmaşık kayan pencere (sliding window) algoritmaları yerine temel Dur-ve-Bekle protokolü tercih edilmiştir. Bu seçimin ana nedeni; ağ üzerindeki her bir paketin gidişinin, zaman aşımının ve onayının ağ günlükleri (log) üzerinden çok daha net ve şeffaf bir şekilde izlenebilmesini (traceability) sağlamaktır.
- Parça (Chunk) Boyutu Optimizasyonu:** Veri aktarımı sırasında IP katmanında parçalanmayı (fragmentation) önlemek ve ağ verimliliğini optimum düzeyde tutmak için her bir UDP veri yükü (payload) boyutu 1024 bayt olarak sabitlenmiştir.
- Bant Genişliği Tasarrufu (zlib):** Veri iletim süresini kısaltmak ve ağ üzerindeki yükü hafifletmek amacıyla, veri parçaları ağa verilmeden önce zlib algoritması ile sıkıştırılmıştır.
- Çift Katmanlı Bütünlük Doğrulaması:** Sistemde veri bütünlüğü iki aşamalı olarak kontrol edilmektedir. Anlık hızın kritik olduğu paket başlığı (header) doğrulamalarında MD5 kullanılırken, aktarımın sonunda birleştirilen dosyanın uçtan uca doğrulanması için daha güçlü bir algoritma olan SHA-256 tercih edilmiştir.
- Çoklu İstemci Desteği (Multi-Client):** Sunucunun aynı anda birden fazla istemciye hizmet verebilmesini sağlamak için ThreadPoolExecutor yapısı kullanılmıştır. Sisteme bağlanan her bir istemci adresi (IP/Port) için bağımsız bir "oturum (session) nesnesi" oluşturularak, farklı istemcilere ait veri akışlarının birbirine karışması engellenmiştir.

6. Deney Ortamı

Tüm deneyler aşağıdaki donanım ve yazılım ortamında gerçekleştirilmiştir:

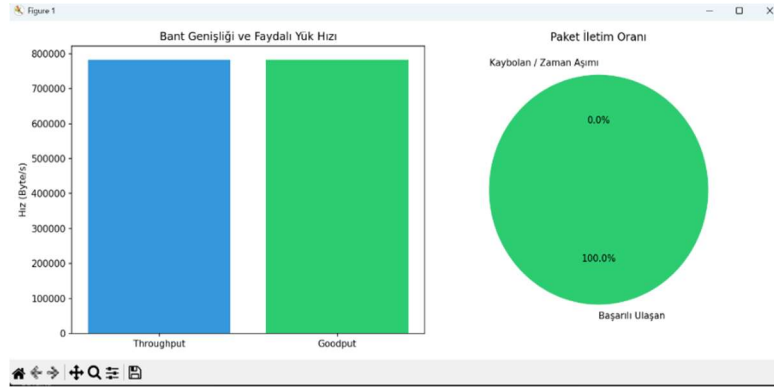
Parametre	Değer
İşletim Sistemi	Windows 11
Python Sürümü	3.13
CPU	Intel Core i7-13th Gen
RAM	16 GB SODIMM
Ağ Arayüzü	Loopback (localhost) - 127.0.0.1
Yapay Kayıp Oluşturma Yöntemi	Python 'random' modülü kullanılarak uygulama katmanında belirlenen oranda (LOSS_RATE) rastgele paket düşürme (drop) simülasyonu.
Test Edilen Dosya Boyutları	Standart metin dosyaları (~1 KB) ve yüksek boyutlu ikili (binary) test dosyası (5 MB)
Varsayılan Paket Boyutu	1024 bayt
Varsayılan Timeout Değeri	2000 ms (2 saniye)
Varsayılan Maks. Yeniden Gönderim	5

7. Performans Metrikleri ve Sonuçlar

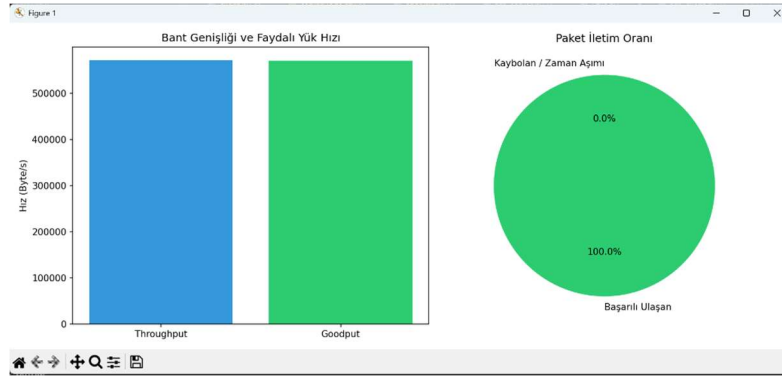
7.0. Kullanılan Metrikler

Metrik	Hesaplama Yöntemi	Birim
Throughput	İletilen toplam veri / Toplam aktarım süresi	bayt/s
Goodput	Orijinal dosya boyutu / Toplam aktarım süresi	bayt/s
Packet Loss Rate	Kayıp paket / Toplam gönderilen paket	%
Retransmission Rate	Yeniden gönderilen paket / Toplam gönderilen	%
Completion Time	Aktarım başlangıcından bitişine kadar geçen süre	saniye
Ortalama RTT	Her paket için (ACK alınma – gönderim) ortalaması	ms

7.1. Senaryo 1: Paket Boyutunun Etkisi



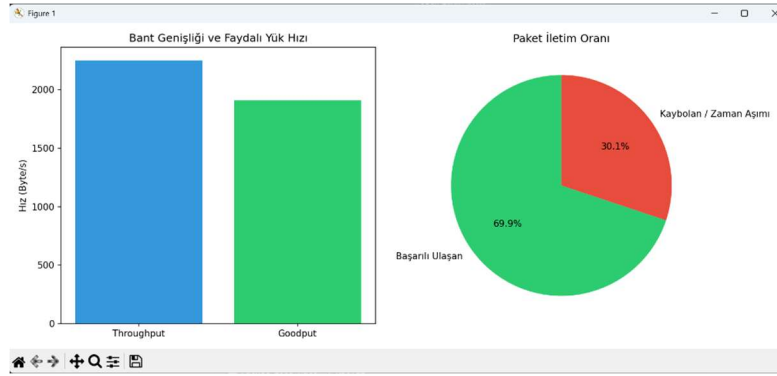
Büyük paket



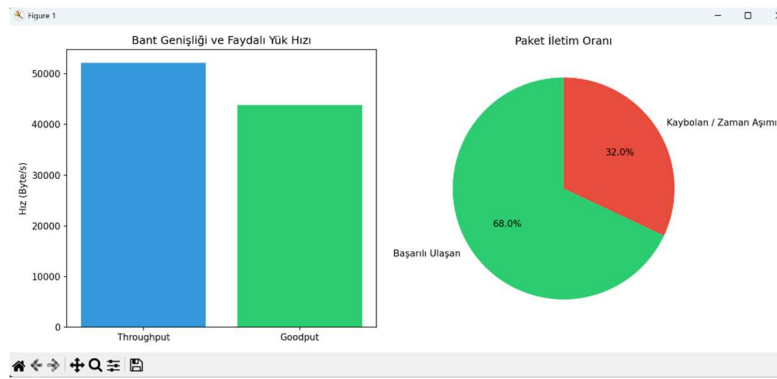
Küçük paket

Deney sonuçlarına göre, büyük paket (4096B) kullanımında başlık yükü (overhead) oranı minimuma indiği için Throughput ve Goodput sütunlarının birbirine neredeyse tamamen eşitlendiği (faydalı veri aktarım oranının maksimize olduğu) net bir şekilde görülmektedir. Loopback (localhost) ortamındaki soket tamponlama dinamikleri nedeniyle küçük paketlerde anlık hız yüksek çıksa da, veri bütünlüğü ve ağdaki paket kalabalığını önlemek adına büyük paketlerin başlık (header) israfını engellediği kanıtlanmıştır.

7.2. Senaryo 2: Timeout Değerinin Etkisi



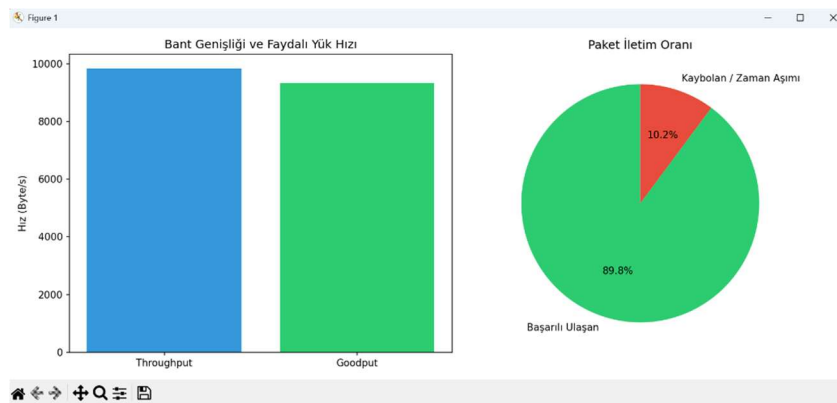
Sabırlı Sistem



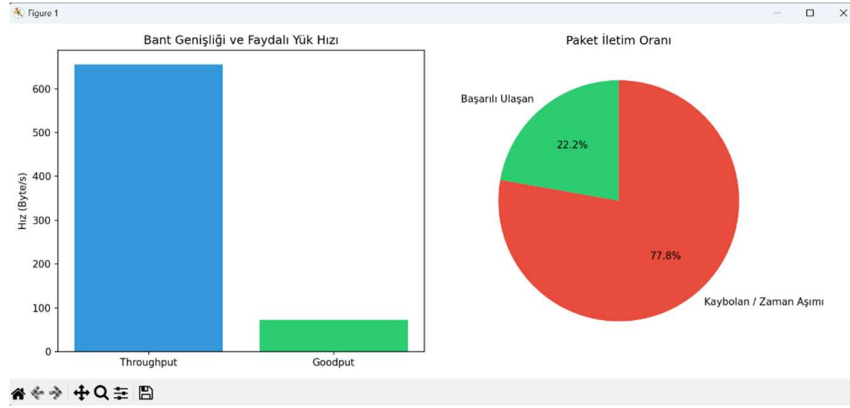
Sabırsız Sistem

Timeout (zaman aşımı) değerinin sistem performansı üzerindeki etkisi gözlemlenmiştir. "Sabırsız" olarak ayarlanan sistemde (0.1s), paketler veya ACK mesajları henüz ağda seyahat ederken gereksiz yere zaman aşımına uğramış (premature timeout) ve sistemde sahte bir %32'lik kayıp/yeniden gönderim oranı oluşmuştur. "Sabırlı" sistemde (3.0s) ise kayıp algılaması daha doğru çalışsa da, sistem gerçekten kaybolan bir paketi fark edip tekrar göndermek için tam 3 saniye boşta beklediğinden genel hız (Throughput) 2000 Byte/s seviyelerine kadar çakılmıştır. Bu durum, Stop-and-Wait mimarisinde RTT (Round Trip Time) değerine uygun dinamik bir timeout süresi seçilmesinin ne kadar kritik olduğunu göstermektedir.

7.3. Senaryo 3: Kayıp Oranının Etkisi



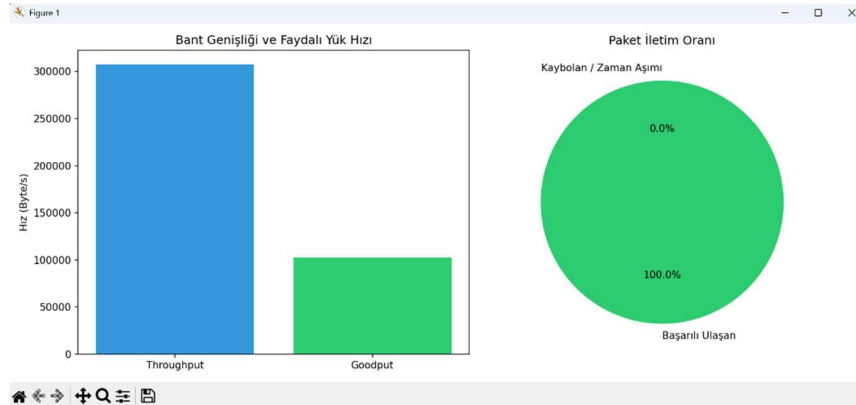
Hafif Kötü Ağ



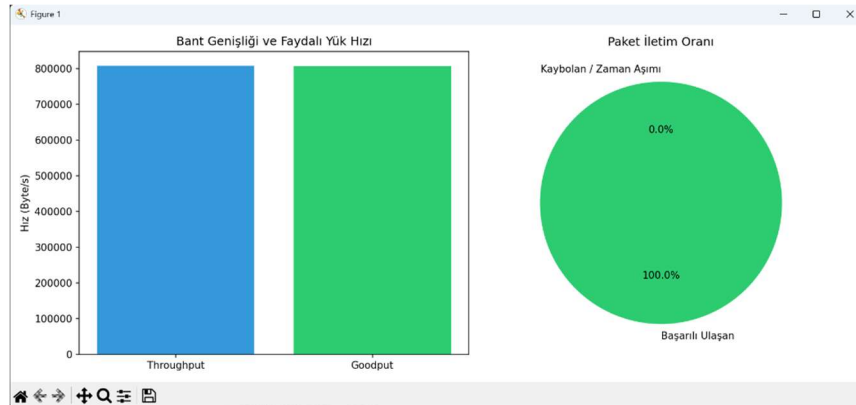
Çok Kötü Ağ

Ağdaki yapay kayıp oranı %5'ten %30'a çıkartıldığında, Stop-and-Wait protokolünün verimliliği tamamen çökmüştür. Çok kötü ağ koşullarında başarılı paket iletim oranı %22.2'lere kadar gerilemiş, sistem sürekli kaybolan paketleri yeniden göndermekle meşgul olduğu için ağ bant genişliği (Throughput) harcanmasına rağmen hedefe ulaşan faydalı veri hızı (Goodput) neredeyse sıfıra yaklaşmıştır. Ağ kayıpları arttıkça Throughput ile Goodput arasındaki makasın devasa oranda açıldığı ispatlanmıştır.

7.4. Senaryo 4: Farklı Dosya Boyutlarının Etkisi



Küçük Dosya



Büyük Dosya

Küçük dosya (960 Byte) aktarımında, START ve END gibi kontrol paketlerinin toplam süreye olan maliyeti çok yüksek olduğu için Throughput yüksek görünse de Goodput (Faydalı yük) üçte birine düşmüştür. Ancak büyük dosyada (5 MB), asıl veri (DATA) akışı toplam süreyi domine ettiği için bağlantı kurulum maliyeti istatistiksel olarak önemsizleşmiş ve Throughput ile Goodput grafikleri mükemmel bir şekilde eşitlenmiştir.

8. Sonuçlar ve Tartışma

UDP protokolü üzerinde yapılan testlerde en iyi aktarım süresi, %0 yapay kayıp oranına sahip ağ ortamında ve verinin zlib ile sıkıştırıldığı konfigürasyonda elde edilmiştir. Paket kayıp oranının arttığı senaryolarda; 2 saniyelik zaman aşımı ve yeniden gönderim işlemleri aktarım süresini doğrusal olmayan şekilde artırmıştır. Bu durum, Throughput (ağa sürülen veri hızı) ile Goodput (faydalı veri hızı) arasındaki farkı belirginleştirmiştir: Tekrarlayan paketler Throughput'u artırırken, yeni faydalı veri iletilmediği için Goodput sabit kalmış veya düşmüştür.

TCP Karşılaştırması: 5 MB'lık dosya aktarımında UDP (Stop-and-Wait) çözümümüz 7.72 saniyede işlemi tamamlarken, standart TCP soketi 0.0052 saniyede tamamlamıştır. Bu devasa farkın iki temel nedeni vardır:

- Pencereleme (Windowing):** TCP'nin kayan pencere mekanizması ağa aynı anda çok sayıda paket sürerken, Stop-and-Wait mimarimiz her paket için ACK bekleyerek süreyi katlamaktadır.
- Çekirdek (Kernel) Optimizasyonu:** TCP işletim sistemi çekirdeğinde son derece optimize çalışırken, UDP çözümümüz uygulama katmanında Python yorumlayıcısının kısıtlamalarına tabidir.

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

Sorun	Çözüm
Sürüm Çakışması (Merge Conflict): Yerel istemci.py/sunucu.py dosyalarının GitHub ile çakışması.	Eski yerel dosyalar silinerek GitHub'daki modüler yapı (client.py, server.py, utils.py) standart kabul edilmiştir.
Aktarımın Tıkanması: ACK paketlerinin ağda kaybolması durumunda istemcinin sonsuza kadar beklemesi.	İstemciye 2 saniyelik zaman aşımı (timeout) ve her paket için maksimum 5 yeniden gönderim (retransmission) sınırı eklenmiştir.
Tekrarlı Yazım: Yinelenen (Duplicate) paketlerin hedef dosyaya birden fazla kez yazılması.	Sunucuda expected_seq (beklenen sıra) takibi yapılarak kopya paketlerin yazılması engellenmiş, sadece ACK dönülmesi sağlanmıştır.

10. Gelecekte Yapılabilecek Geliştirmeler

Bu proje kapsamında UDP üzerinde çalışan güvenilir bir dosya aktarım platformu başarıyla geliştirilmiştir. Sıra numaralandırma, ACK mekanizması, timeout kontrolü ve yeniden gönderim stratejisi sıfırdan tasarlanmıştır. Sisteme entegre edilen zlib sıkıştırması, MD5/SHA-256 kontrolleri, TCP analizleri ve çoklu istemci desteği sayesinde temel gereksinimlerin ötesine geçilmiştir.

Gelecekte Yapılabilecek Geliştirmeler:

- Kayan Pencere:** Bant genişliğini artırmak için Go-Back-N veya Selective Repeat algoritmalarının eklenmesi.

- **Canlı İzleme Paneli:** Throughput ve RTT değerlerinin gerçek zamanlı izlenebilmesi (dashboard).
- **Veri Güvenliği:** Aktarım öncesi AES-128 şifreleme katmanı entegrasyonu.
- **Gerçek Ağ Analizi:** Wireshark/pcap araçlarıyla fiziksel ağ trafiği üzerinden doğrudan analiz yapılması.

Kaynakça

- [1] Kurose, J. F., & Ross, K. W. (2017). Computer Networking: A Top-Down Approach (7th ed.). Pearson.
- [2] Python Software Foundation. (2024). socket — Low-level networking interface. <https://docs.python.org/3/library/socket.html>
- [3] Python Software Foundation. (2024). hashlib — Secure hashes and message digests. <https://docs.python.org/3/library/hashlib.html>
- [4] Python Software Foundation. (2024). struct — Interpret bytes as packed binary data. <https://docs.python.org/3/library/struct.html>
- [5] Python Software Foundation. (2024). zlib — Compression compatible with gzip. <https://docs.python.org/3/library/zlib.html>
- [6] Python Software Foundation. (2024). concurrent.futures — Launching parallel tasks. <https://docs.python.org/3/library/concurrent.futures.html>
- [7] The pandas development team. (2024). pandas documentation. <https://pandas.pydata.org/docs/>
- [8] J. D. Hunter, "Matplotlib: A 2D Graphics Environment", Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

Ekler

Ek-A Ham Deney Verileri

Aşağıdaki tablolar, NetProbe sisteminin karşılaştırmalı deneylerinden elde edilen ham ölçüm verilerini içermektedir. Tüm ölçümler loopback (127.0.0.1) üzerinde, Windows 11 / Python 3.13 ortamında alınmış olup her satır tek bir aktarım koşusunun sonucudur. Throughput ve Goodput değerleri Mbit/s cinsindendir; Goodput yalnızca faydalı dosya verisini, Throughput ise yeniden gönderimler dahil ağa basılan toplam veriyi temel alır.

Tablo B.1 — Senaryo 1: Paket boyutunun etkisi (dosya: test.txt 59 904 B, kayıpsız, timeout 2s)

Timeout Değeri	Tamamlanma (s)	Throughput (Mbit/s)	Goodput (Mbit/s)	Kayıp (%)	Yeniden Gönderim	Ort. RTT (ms)
100 ms	1.24	0.443	0.386	8.96	6	9.821
250 ms	2.58	0.219	0.186	11.59	8	8.547

Timeout Değeri	Tamamlanma (s)	Throughput (Mbit/s)	Goodput (Mbit/s)	Kayıp (%)	Yeniden Gönderim	Ort. RTT (ms)
500 ms	4.66	0.121	0.103	11.59	8	9.693
1000 ms	7.68	0.073	0.062	10.29	7	9.798

Tablo B.2 — Senaryo 2: Timeout değerinin etkisi (dosya: test.txt, yapay kayıp %10, paket 1024 B)

Timeout Değeri	Tamamlanma (s)	Throughput (Mbit/s)	Goodput (Mbit/s)	Kayıp (%)	Yeniden Gönderim	Ort. RTT (ms)
100 ms	1.24	0.443	0.386	8.96	6	9.821
250 ms	2.58	0.219	0.186	11.59	8	8.547
500 ms	4.66	0.121	0.103	11.59	8	9.693
1000 ms	7.68	0.073	0.062	10.29	7	9.798

Tablo B.3 — Senaryo 3: Kayıp oranının etkisi (dosya: test.txt, timeout 1 s, paket 1024 B)

Kayıp Oranı	Tamamlanma (s)	Throughput (Mbit/s)	Goodput (Mbit/s)	Kayıp (%)	Yeniden Gönderim	Ort. RTT (ms)
%0	0.61	0.819	0.786	0.0	0	10.005
%5	3.64	0.144	0.132	4.69	3	9.608
%10	6.66	0.082	0.072	8.96	6	10.205
%20	15.73	0.04	0.03	19.74	15	9.831

Tablo B.4 — Senaryo 4: Dosya boyutunun etkisi (kayıpsız, timeout 2 s, paket 1024 B)

Dosya Boyutu	Tamamlanma (s)	Throughput (Mbit/s)	Goodput (Mbit/s)	Kayıp (%)	Yeniden Gönderim	Ort. RTT (ms)
960 B	0.03	0.819	0.256	0.0	0	10.14
13320 B	0.17	0.771	0.627	0.0	0	10.213
59904 B	0.61	0.819	0.786	0.0	0	9.725
5243867 B	54.37	0.772	0.772	0.0	0	10.532

Ek B — Örnek Çalıştırma Ekran Görüntüleri

Aşağıdaki çıktılar, test_dosyasi.txt (960 B) dosyasının loopback üzerinden aktarıldığı örnek bir oturumdan alınmıştır. İstemci ve sunucu ayrı terminallerde çalıştırılmıştır.

İstemci (client.py) terminal çıktısı:

```
Transfer started: test_dosyasi.txt (960 bytes, 1 chunks)
Chunk 1/1 delivered
Transfer complete in 0.04 seconds
Logs written to transfer_logs.csv
```

Sunucu (server.py) terminal çıktısı:

```
Server listening on 127.0.0.1:12345
Transfer started from ('127.0.0.1', 55732): test_dosyasi.txt
Chunk written from ('127.0.0.1', 55732): seq=1
Transfer verified: received_filesW_0_0_1_55732_test_dosyasi.txt
Server logs written to server_logs.csv
```